

UNIT 1 — Collective Communication Operations

Q1) Scatter and Gather Communication Operation. SRDESRUUMC - RMRRUSRIUMC

- Scatter and Gather are collective communication operations in parallel computing.
- They are used for distributing and collecting data among processors.
- These operations improve workload distribution and parallel execution.

Scatter:

1. Scatter operation starts from a root processor.
2. Root processor divides data into multiple parts.
3. Different data blocks are sent to different processors.
4. Each processor receives only its assigned portion.
5. Scatter supports parallel processing of large datasets.
6. It reduces computation load on a single processor.
7. Scatter is commonly used before parallel computation.
8. Example: matrix rows distributed among processors.
9. Useful in matrix operations, image processing, and scientific simulations.
10. MPI provides the function MPI_Scatter() for this operation.
11. Communication pattern is one-to-many.

Gather:

1. Gather is the reverse operation of Scatter.
2. Multiple processors send computed results to the root processor.
3. Root processor collects all partial outputs.
4. Results are combined into a single dataset.
5. Used after parallel execution is completed.
6. Supports result aggregation and consolidation.
7. Reduces complexity of collecting distributed results.
8. Improves efficiency in parallel applications.
9. Useful in numerical computation, data analytics, and scientific computing.
10. MPI provides the function MPI_Gather() for this operation.
11. Communication pattern is many-to-one.

Q2) One-to-All Broadcast? Cost Calculation. CSU-OISANRMBACI

- One-to-All Broadcast is a collective communication operation used in parallel computing.
 - In this operation, a single source processor sends the same message to all other processors in the system.
 - It is commonly used when all processors require identical input data before starting computation.
1. One processor acts as the source node.
 2. Initially, only the source processor contains the message.
 3. The source sends the message to one neighboring processor.
 4. After receiving the message, the receiver also becomes a sender.
 5. The number of informed processors doubles after every communication step.
 6. Recursive doubling is used to minimize communication time.
 7. Multiple communications occur simultaneously in each step.
 8. Broadcast continues until every processor receives the message.
 9. The algorithm scales well as the number of processors increases.
 10. Communication load is distributed among processors rather than remaining with the source.

11. This significantly improves performance in large parallel systems.

Q3) All-to-All Broadcast Communication Operation. CMAMU-EEECLNCSUSICS

- All-to-All Broadcast is a collective communication operation used in parallel and distributed computing.
 - It is a many-to-many communication pattern where every processor sends its data to all other processors.
 - After completion, every processor possesses data from every other processor in the system.
 - It is one of the most communication-intensive operations because all processors participate simultaneously.
 - All-to-All Broadcast is widely used in scientific computing, distributed databases, parallel sorting, and large-scale data processing applications.
1. Each processor owns unique data.
 2. Every processor sends data to all others.
 3. Every processor receives data from all others.
 4. Communication occurs simultaneously.
 5. Large amount of message exchange takes place.
 6. Network utilization becomes high.
 7. Communication complexity increases with processors.
 8. Synchronization is required.
 9. Used in distributed databases, scientific simulations, sorting algorithms.
 10. Supports global information sharing.
 11. Improves data availability.
 12. Communication overhead is high.
 13. Efficient interconnection networks are preferred.
 14. Scalability depends on communication cost.

Q4) All-to-One Broadcast and Reduction on a Ring. CRRP-OEDMIECR-ERPIFCRC

- All-to-One Broadcast is a collective communication operation in which data from all processors is collected at a single destination processor.
- Reduction is a related operation where data from multiple processors is combined using an arithmetic or logical operation to produce a single result.
- Ring topology is commonly used to implement these operations because of its simple structure and communication mechanism.
- In a ring network, processors are connected in a circular fashion, and messages are passed through neighboring processors.

Working of All-to-One Broadcast on a Ring

1. One processor acts as the destination (root) node.
2. Every processor contributes its local data.
3. Data is forwarded through neighboring processors.
4. Messages move around the ring one step at a time.
5. Intermediate processors relay received messages.
6. Eventually, all data reaches the root processor.
7. Communication requires multiple transmission steps.
8. The root processor collects information from every node.

Reduction on a Ring

1. Each processor initially contains a local value.
2. Reduction combines these values into a single result.
3. Partial results are generated during message transmission.
4. Intermediate processors perform reduction operations before forwarding data.
5. Only one final result reaches the root processor.
6. Communication and computation occur simultaneously.
7. Reduction reduces the amount of data transferred.

8. Common reduction operations include: Sum, Maximum, Minimum, Product, Logical AND, Logical OR.

Q5) Blocking and Non-Blocking Communication Using MPI. SESU-CSRCEPEESIL-CSRCBSCH

- MPI (Message Passing Interface) is a standard communication library used in parallel and distributed computing.
- It enables processes running on different processors to exchange data through message passing.
- MPI supports two major communication modes: Blocking Communication & Non-Blocking Communication.
- These communication methods are used to coordinate parallel tasks and exchange information efficiently.

Blocking Communication:

1. Blocking communication is a communication method in which the sending or receiving process waits until the communication operation is completed.
2. Sender waits until the message is safely transferred.
3. Receiver waits until the message arrives completely.
4. Communication is synchronous in nature.
5. Execution is suspended during communication.
6. Processor cannot perform other tasks while waiting.
7. Easy to understand and implement.
8. Easier debugging and error detection.
9. Suitable for simple parallel programs.
10. May lead to processor idle time.
11. Lower resource utilization compared to non-blocking communication.

Non-Blocking Communication:

1. Non-blocking communication allows a process to initiate communication and immediately continue executing other tasks without waiting for the communication to finish.
2. Sender starts communication and continues computation.
3. Receiver can perform useful work while waiting for data.
4. Communication and computation overlap.
5. Better processor utilization.
6. Reduces idle time.
7. Improves parallel efficiency.
8. Suitable for large-scale parallel applications.
9. More complex than blocking communication.
10. Higher resource utilization compared to blocking communication.

Q6) Prefix-Sum Operation. MCKUH-IFSTPSIREPC

- Prefix-Sum is one of the most important communication and computation primitives in parallel computing.
 - It computes cumulative results of a sequence of elements.
 - It is also known as the Scan Operation.
 - Prefix-Sum is widely used in many parallel algorithms.
 - It helps in efficient data aggregation and parallel processing.
1. Input consists of an ordered sequence of elements.
 2. First output element remains unchanged.
 3. Second output contains the sum of the first two elements.
 4. Third output contains the sum of the first three elements.
 5. The process continues cumulatively for all elements.
 6. Also known as the Scan operation.
 7. Supports efficient parallel computation.
 8. Can be implemented using recursive doubling techniques.
 9. Reduces overall execution time compared to sequential computation.
 10. Efficiently implemented on hypercube and tree architectures.
 11. Provides cumulative aggregation of distributed data.

12. Can perform operations other than addition such as maximum, minimum, and multiplication.

Q7) Circular Shift Operation on Mesh and Hypercube Network. IUELUH-EDLNCFCDN-CPCFCDS

- Circular Shift is an important communication operation in parallel computing systems.
 - It is used to cyclically transfer data among processors.
 - In this operation, each processor sends data to another processor in a fixed pattern.
 - The last processor sends its data back to the first processor, forming a circular movement.
 - Circular shift is widely used in parallel matrix multiplication, sorting algorithms, load balancing, and data redistribution.
 - It helps achieve efficient communication and workload distribution among processors.
1. Every processor sends data to a neighbouring processor.
 2. Data movement occurs in a circular manner.
 3. Last processor sends data back to first processor.
 4. No processor loses data during transfer.
 5. Circular shift supports load balancing.
 6. It is frequently used in matrix multiplication.
 7. Mesh network contains processors arranged in rows and columns.
 8. In mesh, data moves horizontally or vertically.
 9. Neighbouring mesh nodes exchange information.
 10. Hypercube network consists of nodes connected by dimensions.
 11. Hypercube provides multiple communication paths.
 12. Circular shift in hypercube occurs dimension-wise.
 13. Hypercube communication is faster than mesh.
 14. Communication overhead is reduced.
 15. Data redistribution becomes efficient.
 16. Scalability improves in large parallel systems.

UNIT 2 — Parallel Performance & Matrix Algorithms

Q1) Parallel Matrix-Matrix Multiplication Algorithm with Example. MUDHR-TMCMRCLNPRCPSE

- Matrix multiplication is one of the most important and widely used operations in parallel computing.
- It is extensively used in scientific computing, graphics, machine learning, and data analysis.
- Parallel Matrix-Matrix Multiplication divides the computation among multiple processors.
- Matrix multiplication is highly suitable for parallelization.
- Parallel execution significantly reduces computation time and improves processor utilization.

Matrix Multiplication:

Given two matrices: $A(m \times n)$ and $B(n \times p)$. Their product $C = A \times B$, where each element: $C(i,j) = \sum A(i,k) \times B(k,j)$

1. Two matrices A and B are multiplied to obtain matrix C.
2. Matrix element $C(i,j)$ is computed using row-column multiplication.
3. Computation of different elements is independent.
4. Matrix data can be partitioned among processors.
5. Rows of matrix A are distributed to processors.
6. Columns of matrix B are made available for multiplication.
7. Local computations are performed simultaneously.
8. No dependency exists among independent matrix elements.
9. Partial results are generated by individual processors.
10. Results are gathered after computation completes.
11. Communication is required for data distribution and result collection.
12. Proper load balancing improves efficiency.
13. Speedup increases with the number of processors.
14. Execution time decreases significantly compared to sequential execution.

Parallel Matrix Multiplication Algorithm:

Step 1: Divide matrix A among available processors.

Step 2: Distribute or broadcast the required rows/columns of matrices.

Step 3: Each processor computes assigned elements of matrix C independently.

Step 4: Processors perform local multiplications and additions.

Step 5: Partial results are generated simultaneously.

Step 6: Computed results are gathered to form the final matrix C.

Q2) Granularity and Effects on Performance of Parallel Systems. RDS-GFCFCCSLBP

- Granularity represents the amount of computation between communication events.
 - It determines communication-to-computation ratio.
 - Granularity significantly affects parallel performance.
1. Granularity is classified as Fine Grain and Coarse Grain.
 2. Fine grain has small computation and frequent communication.
 3. Coarse grain has large computation and less communication.
 4. Fine grain provides high parallelism, improves load balancing, increases communication overhead & may reduce efficiency.
 5. Coarse grain reduces communication cost, improves processor utilization & may create load imbalance.
 6. Communication time affects overall performance.
 7. Computation time should dominate communication time.
 8. Very small granularity causes excessive overhead.
 9. Very large granularity reduces parallelism.

10. Balanced granularity provides best performance.
11. Choosing proper granularity improves speedup and efficiency.

Q3) Sources of Overhead in Parallel Systems.

- Overhead is the additional work performed during parallel execution that does not directly contribute to solving the problem.
- It represents extra time and resources consumed because of parallelization.
- Overhead reduces the benefits of parallel processing by increasing execution time.
- High overhead decreases speedup, efficiency, and scalability.
- Therefore, minimizing overhead is essential for achieving high performance in parallel systems.

Sources of Overhead in Parallel Systems

1. Communication Overhead: Communication overhead occurs when processors exchange data and messages during execution.

Causes: Message passing between processors. | Data transfer across the network. | Sending and receiving operations. | Data distribution and result collection.

Effects: Increases execution time. | Causes processors to wait for data. | Reduces overall efficiency.

2. Synchronization Overhead: Synchronization overhead occurs when processors must coordinate their activities.

Causes: Barrier synchronization. | Locks and semaphores. | Mutual exclusion operations. | Waiting for all processors to reach a common point.

Effects: Some processors remain idle. | Execution delays occur. | Efficiency decreases.

3. Load Imbalance Overhead: Load imbalance occurs when work is not distributed evenly among processors.

Causes: Unequal task allocation. | Different computation times. | Irregular workloads.

Effects: Some processors finish early. | Idle processors waste resources. | Reduced speedup and efficiency.

4. Extra Computation Overhead: Additional computations required to manage parallel execution.

Causes: Task scheduling. | Data partitioning. | Communication management. | Parallel control operations.

Effects: Additional CPU cycles are consumed. | Execution time increases. | Reduces overall performance.

5. Memory Overhead: Memory overhead refers to additional memory required for parallel execution.

Causes: Extra communication buffers. | Temporary storage. | Replicated data structures. | Shared-memory management.

Effects: Increased memory consumption. | Reduced memory efficiency. | Additional memory management cost.

6. Process Creation Overhead: Time required to create and initialize processes or threads.

Causes: Process creation. | Thread creation. | Resource allocation. | Initialization activities.

Effects: Delays before actual computation starts. | Additional system resource consumption.

7. Context Switching Overhead: Context switching occurs when the CPU switches from one process or thread to another.

Causes: Multitasking. | Thread scheduling. | Operating system interrupts.

Effects: CPU spends time saving and restoring states. | Execution delays occur. | Performance decreases.

8. I/O Overhead: Time spent performing input/output operations.

Causes: Reading files. | Writing results. | Accessing storage devices.

Effects: Slows computation. | Increases total execution time.

Impact of Overhead on Performance

1. Reduces Speedup: More overhead means less improvement from parallelism.
2. Reduces Efficiency: Processors spend time on non-productive activities.
3. Increases Execution Time: Additional work delays program completion.
4. Limits Scalability: Adding more processors may increase overhead instead of improving performance.
5. Increases Resource Consumption: Extra memory, CPU time, and network bandwidth are required.

Methods to Reduce Overhead

Minimize communication among processors. | Use efficient synchronization mechanisms. | Balance workload evenly. | Reduce process creation frequency. | Optimize memory usage. | Use larger task granularity. | Overlap communication and computation. | Employ efficient scheduling algorithms.

Q4) Minimum Execution Time and Minimum Cost Optimal Execution Time

- Execution time and cost are two important performance measures of parallel algorithms.
- The primary goal of parallel computing is to reduce program execution time by utilizing multiple processors.
- However, merely reducing execution time is not sufficient; processors must also be used efficiently.
- A good parallel algorithm should achieve both minimum execution time and cost optimality.

1. Minimum Execution Time

It focuses on reducing program completion time. Workload is distributed among multiple processors. Parallel execution allows multiple tasks to run simultaneously. Increasing the number of processors generally reduces execution time.

Speedup is achieved by performing computations concurrently.

Communication and synchronization overhead influence execution time.

Excessive communication may limit performance gains.

Processor idle time should be minimized.

The objective is to achieve the fastest possible execution.

2. Minimum Cost Optimal Execution Time

Cost optimality measures resource efficiency.

It prevents wastage of processors.

Processor utilization should remain high.

No processor should remain unnecessarily idle.

Total computational effort should remain close to sequential work.

Cost optimal algorithms balance speed and efficiency.

Increasing processors should provide proportional benefits.

Communication overhead should be minimized.

Synchronization delays should be reduced.

Cost optimality improves scalability.

Q5) Dense Matrix Algorithms

(i) Matrix Vector Multiplication

- Matrix-vector multiplication is a fundamental dense matrix operation.
- It is widely used in machine learning and scientific computing.

- Parallel execution improves performance.

1. Matrix A is multiplied with vector X.
2. Result is stored in vector Y.
3. Formula is $Y = A \times X$.
4. Rows of matrix are distributed among processors.
5. Each processor computes assigned rows.
6. Computations are independent.
7. Communication occurs during distribution.
8. Local multiplication is performed.
9. Partial results are generated.
10. Results are collected after computation.
11. Load balancing improves efficiency.
12. Parallel execution reduces completion time.
13. Memory usage is optimized.
14. Suitable for large matrices.
15. Frequently used in numerical computing.
16. Provides good speedup.

(ii) Matrix-Matrix Multiplication

- Matrix-Matrix Multiplication is a dense matrix algorithm.
 - It involves multiplication of two large matrices.
 - Parallel implementation significantly improves performance.
1. Two matrices A and B are multiplied.
 2. Result matrix C is generated.
 3. Computation of each element is independent.
 4. Matrix is partitioned among processors.
 5. Row-wise partitioning may be used.
 6. Block partitioning may be used.
 7. Processors perform local multiplication.
 8. Multiple computations occur simultaneously.
 9. Communication distributes matrix data.
 10. Synchronization combines partial results.
 11. Load balancing improves utilization.
 12. Speedup increases with processors.
 13. Large matrices benefit most.
 14. Communication overhead should be minimized.
 15. Common in scientific applications.
 16. Considered one of the most important parallel algorithms.

UNIT 3 — CUDA Programming

Q1) What is CUDA? Draw and Explain CUDA Architecture in Detail.

- CUDA stands for Compute Unified Device Architecture.
- It is a parallel computing platform and programming model developed by NVIDIA.
- CUDA enables programmers to use GPU resources for General Purpose Computing (GPGPU).
- It allows computationally intensive tasks to be executed in parallel on thousands of GPU cores.
- CUDA is widely used in machine learning, scientific computing, image processing, simulations, and high-performance computing (HPC).
- It significantly improves execution speed compared to CPU-only execution.

Components of CUDA Architecture

1. Host (CPU) — The CPU is called the Host in CUDA architecture.

Functions: Executes sequential code. | Controls program execution. | Allocates memory. | Launches kernels on the GPU. | Transfers data between CPU and GPU. | Manages input and output operations.

2. Device (GPU) — The GPU is called the Device.

Functions: Executes parallel portions of programs. | Performs large-scale computations. | Executes thousands of threads simultaneously.

3. PCIe Bus — PCIe (Peripheral Component Interconnect Express) connects the Host and Device.

Functions: Transfers data between CPU and GPU. | Enables kernel launch requests. | Supports memory exchange.

4. Kernel Function — A Kernel is a function executed on the GPU.

Characteristics: Launched by the CPU. | Executed by thousands of threads. | Performs parallel computations.

5. Grid — A Grid is a collection of thread blocks executing a kernel.

Characteristics: Created when a kernel is launched. | Contains one or more blocks. | Enables large-scale parallelism.

6. Thread Block — A Block is a group of threads.

Characteristics: Threads within a block cooperate. | Threads share shared memory. | Synchronization is possible within a block. | Blocks execute independently.

7. Threads — Threads are the smallest execution units in CUDA.

Characteristics: Execute instructions independently. | Each thread has a unique ID. | Thousands of threads can execute simultaneously. | Responsible for actual computation.

8. Streaming Multiprocessor (SM) — SM is the basic processing unit of a GPU.

Functions: Executes thread blocks. | Schedules warps. | Manages shared memory.

9. CUDA Cores — CUDA Cores are arithmetic processing units inside an SM.

Functions: Perform mathematical operations. | Execute thread instructions. | Process floating-point and integer calculations.

Q2) Different Programming Languages Support in CUDA.

- CUDA supports multiple programming languages allowing developers from different backgrounds to use GPU computing.
- NVIDIA provides official APIs, libraries, and compiler support for each of these languages.

- Language support makes CUDA accessible for scientific computing, data science, and machine learning communities.

CUDA Programming Language Support:

1. CUDA C/C++ (Primary Language)

Native language for CUDA development. | Extends C/C++ with special keywords: `__global__`, `__device__`, `__host__`. | Supports templates, classes, and C++17 features on GPU. | Compiled using NVCC (NVIDIA CUDA Compiler). | NVCC splits host (CPU) and device (GPU) code automatically.

2. CUDA Fortran

Supported via NVIDIA HPC SDK (formerly PGI compiler). | Used in scientific & HPC applications. | Uses CUF kernel directives to launch GPU kernels. | Helps accelerate legacy scientific codebases without full rewrites.

3. Python — PyCUDA & Numba

PyCUDA -> Python bindings to CUDA API; write CUDA C kernels as strings, compiled at runtime. | Numba -> Uses `@cuda.jit` decorator to JIT-compile Python into CUDA kernels. | No C code needed with Numba. | Ideal for data science, ML, and rapid prototyping.

4. Java & Scala (JVM Languages)

JCuda -> Java bindings for CUDA runtime & driver APIs. | Aparapi -> Converts Java bytecode to CUDA/OpenCL kernels. | Used in Apache Spark (via RAPIDS for GPU acceleration). | Slightly slower than C/C++ due to JVM overhead.

5. MATLAB

Uses Parallel Computing Toolbox for CUDA GPU support. | `gpuArray()` transparently offloads data to GPU. | Custom CUDA kernels called via `feval()` with `.ptx` files. | Popular in academic research, DSP, and image processing.

6. R Language

`gpuR` and `gputools` packages provide CUDA support. | Accelerates matrix operations, SVD, linear algebra. | Used in statistical computing and ML research.

7. .NET (C# / F#)

ManagedCUDA and Cudaify.NET provide GPU bindings. | Enables CUDA programming within the .NET ecosystem. | Useful for Windows enterprise apps needing GPU power.

8. OpenCL Interoperability

CUDA supports interoperability with OpenCL. | Libraries like ArrayFire abstract over both backends. | Helps write portable GPU code for non-NVIDIA hardware too.

Q3) Applications of CUDA.

- CUDA has transformed high-performance computing by enabling GPU acceleration across diverse fields.
 - Its ability to run thousands of parallel threads makes it ideal for data-intensive and compute-heavy applications.
 - Industries from healthcare to finance and entertainment actively use CUDA-accelerated computing.
 - Because GPUs have thousands of cores, they outperform CPUs in tasks that are massively parallel.
 - CUDA has become the foundation of modern AI, science, and engineering computing.
1. Deep Learning and AI — Training neural networks (CNN, RNN, Transformers) uses massive matrix operations that CUDA accelerates thousands of times over CPU. Frameworks like TensorFlow, PyTorch, and Keras use CUDA as their GPU backend.
 2. Image and Video Processing — Real-time filtering, edge detection, compression, and encoding on millions of pixels simultaneously. CUDA processes each pixel independently in parallel, making it ideal for image workloads.

3. Medical Imaging — CT scan reconstruction, MRI processing, and real-time ultrasound imaging use CUDA for faster signal processing.
4. Cryptography — Password cracking, hash computation, and encryption/decryption algorithms are parallelized using CUDA.
5. Computer Vision — Object detection, facial recognition, and autonomous driving use CUDA-accelerated inference.
6. Seismic Data Processing — Oil and gas industry uses CUDA to process seismic survey data for underground analysis.
7. Financial Computing — Risk analysis, Monte Carlo simulations, and options pricing are parallelized using CUDA.
8. Database Acceleration — GPU databases like RAPIDS cuDF accelerate SQL queries and data analytics using CUDA.
9. Natural Language Processing — Training large language models (GPT, BERT) requires CUDA-powered GPU clusters.
10. Robotics and Autonomous Systems — Real-time sensor fusion, SLAM (mapping), and path planning are CUDA-accelerated.

Q4) Advantages and Limitations of CUDA.

Advantages:

1. Massive Parallelism — Modern GPUs have thousands of CUDA cores; all execute threads simultaneously giving huge speedup.
2. High Throughput — CUDA handles thousands of operations per clock cycle, far more than any CPU.
3. Easy to Learn — CUDA C is an extension of C/C++, so C programmers can quickly adapt.
4. Large Developer Community — Extensive documentation, tutorials, and NVIDIA support make learning and debugging easier.
5. High Memory Bandwidth — GPU global memory bandwidth (e.g., 900 GB/s on A100) far exceeds CPU memory bandwidth.
6. Rich Library Ecosystem — cuBLAS (linear algebra), cuDNN (deep learning), cuFFT (signal processing) — ready-made optimized libraries.
7. Scalability — Same CUDA code runs on entry-level GPUs and high-end data center GPUs.
8. Cost Effective — One NVIDIA GPU can replace many CPU nodes for compute-intensive tasks, reducing infrastructure cost.

Limitations:

1. Vendor Lock-in — CUDA only works on NVIDIA GPUs; code cannot run on AMD or Intel GPUs.
2. High Data Transfer Overhead — Copying data between CPU (host) and GPU (device) via PCIe bus is slow and can bottleneck performance.
3. Limited GPU Memory — GPU VRAM (e.g., 16-80 GB) is much smaller than CPU RAM; large datasets may not fit.
4. Not Suitable for Sequential Tasks — CUDA is only beneficial for data-parallel problems; purely sequential algorithms get no speedup.
5. Complex Debugging — Debugging GPU code is harder than CPU code; race conditions in shared memory are difficult to trace.
6. Learning Curve for Optimization — Writing correct CUDA code is easy, but writing high-performance CUDA (memory

coalescing, warp divergence) is complex.

7. Power Consumption — High-end GPUs consume 300-700 watts, increasing operational costs.
8. Thread Divergence — If threads in a warp take different execution paths (if-else), they execute serially, reducing efficiency.

Q5) Processing Flow of a CUDA-C Program (with CUDA C Functions)

- A CUDA-C program runs partly on the CPU (host) and partly on the GPU (device).
- The CPU controls the overall flow — it sets up data, launches the GPU kernel, and collects results.
- The GPU executes the parallel portion of the program using thousands of threads.

Phase 1 — Host Setup (CPU): Program starts on the CPU (host).

malloc() allocates memory in CPU RAM for input/output arrays. | Host data is initialized with input values on the CPU.

Phase 2 — Device Memory Allocation (GPU):

cudaMalloc() allocates memory in GPU VRAM. | Separate memory spaces exist for CPU and GPU — they cannot directly share data.

Phase 3 — Host -> Device Transfer:

cudaMemcpy(dst, src, size, cudaMemcpyHostToDevice) copies input data from CPU RAM -> GPU VRAM. | This transfer happens over the PCIe bus.

Phase 4 — Kernel Configuration & Launch:

Developer defines grid dimensions (number of blocks) and block dimensions (threads per block). | Kernel is launched using the <<>> syntax: myKernel<<>>(args). | Kernel launch is asynchronous — CPU does not wait by default.

Phase 5 — Kernel Execution (GPU):

Thousands of threads execute in parallel on GPU cores. | Each thread computes threadIdx and blockIdx to find its unique data index. | Threads are grouped into warps (32 threads) for execution on SMs (Streaming Multiprocessors).

Phase 6 — Synchronization:

cudaDeviceSynchronize() makes the CPU wait until all GPU threads finish. | Ensures results are fully computed before being read back.

Phase 7 — Device -> Host Transfer:

cudaMemcpy(dst, src, size, cudaMemcpyDeviceToHost) copies results from GPU VRAM -> CPU RAM.

Phase 8 — Host Cleanup:

CPU processes/uses the output results. | cudaFree() releases GPU memory. | free() releases CPU memory. | Program ends.

Q6) Terms in CUDA: Device, Host, Device Code, Kernel.

- These four terms form the foundation of CUDA programming terminology.
- Understanding each term clearly is necessary before writing any CUDA program.
- Every CUDA program is split between Host (CPU) side and Device (GPU) side.

Host:

Host refers to the CPU and its associated system memory (RAM). | Host executes the sequential part of the program. | Host is responsible for allocating GPU memory, transferring data, and launching kernels. | Host code is standard C/C++ compiled by a normal compiler (gcc/msvc). | Host memory is accessed using standard pointers (e.g., int *h_array). | Host cannot directly access GPU memory — must use cudaMemcpy to transfer.

Device:

Device refers to the GPU and its associated video memory (VRAM). | Device executes the parallel part of the program. | Device memory is allocated using `cudaMalloc()` and freed using `cudaFree()`. | Device pointers (e.g., `int *d_array`) point to GPU memory locations. | Device cannot access CPU memory directly — data must be explicitly transferred. | Device contains thousands of CUDA cores organized inside Streaming Multiprocessors (SMs).

Device Code:

Device code is the portion of the CUDA program that runs on the GPU. | It is identified by qualifiers: `__global__` (kernel), `__device__` (helper function on GPU), `__shared__` (shared memory variable). | Device code is compiled by `nvcc` (NVIDIA CUDA Compiler) into PTX bytecode, then to GPU machine code. | Device code cannot call standard CPU library functions (like `printf` in older CUDA versions). | `__device__` functions can only be called from other device or kernel functions, not from CPU.

Kernel:

A Kernel is a special function declared with `__global__` that runs on the GPU. | It is called from the CPU (host) using the `<<>>` syntax. | When launched, the kernel runs as thousands of identical threads in parallel. | Each thread executes the same kernel code but operates on different data (identified by thread ID). | Kernels are asynchronous by default — CPU continues executing after kernel launch unless `cudaDeviceSynchronize()` is called. | Kernels cannot return values directly; results must be written to device memory and then copied to host.

Q7) CUDA Memory Model. Thread Hierarchy.

- CUDA has a rich memory hierarchy designed to balance speed, capacity, and scope of access.
- Different memory types serve different purposes and have very different performance characteristics.
- Understanding the memory model is key to writing efficient CUDA programs.

Memory Model:

1. Registers — Fastest memory, private to each thread; compiler automatically assigns scalar variables to registers; very limited in count.
2. Local Memory — Private to each thread but physically stored in global memory (slow); used when registers overflow (register spilling).
3. Shared Memory — Fast on-chip memory (~5 cycle latency); shared by all threads within the same block; declared with `__shared__`; used for inter-thread communication within a block.
4. Global Memory — Largest GPU memory (GBs of VRAM); accessible by all threads in all blocks; slowest (~400-600 cycle latency); declared using `cudaMalloc()`.
5. Constant Memory — Read-only, 64KB, cached; all threads read the same value efficiently; declared with `__constant__`; good for kernel parameters and lookup tables.
6. Texture Memory — Read-only, cached, optimized for 2D spatial access patterns; useful for image processing workloads.

Thread Hierarchy:

1. Thread — The smallest execution unit in CUDA; each thread runs one instance of the kernel independently.
2. Each thread has built-in variables: `threadIdx.x`, `threadIdx.y`, `threadIdx.z` for its local position within a block.
3. Thread Block — A group of threads (up to 1024 threads per block) that execute together on the same SM.

4. Threads within a block can share data via shared memory and synchronize using `__syncthreads()`.
5. Each block has an ID: `blockIdx.x`, `blockIdx.y`, `blockIdx.z`.
6. Grid — A collection of all blocks launched for one kernel; all blocks execute the same kernel.
7. Grid can be 1D, 2D, or 3D depending on the problem structure.
8. Global thread ID formula: $tid = threadIdx.x + blockIdx.x * blockDim.x$.

Q8) Block Dimension and Grid Dimension in CUDA.

- Block dimension and grid dimension are the two key parameters specified when launching a CUDA kernel.
- Together they define the total number of threads created and how they are organized.
- Choosing the right dimensions is critical for performance and correctness.

Block Dimension (blockDim):

1. Block dimension defines the number of threads inside a single thread block.
2. Specified as a `dim3` structure with x, y, z components: `dim3 blockDim(x, y, z)`.
3. Maximum 1024 threads per block total: $x * y * z \leq 1024$.
4. For 1D problems: `dim3 blockDim(256, 1, 1)` means 256 threads in a block.
5. For 2D problems (e.g., image): `dim3 blockDim(16, 16, 1)` means $16 \times 16 = 256$ threads per block.
6. Inside the kernel, `blockDim.x` gives the number of threads in x-direction of a block.
7. All threads within a block share the same Shared Memory and can synchronize with `__syncthreads()`.
8. Block size should be a multiple of 32 (warp size) for best GPU utilization.

Grid Dimension (gridDim):

1. Grid dimension defines the number of blocks in the entire grid.
2. Also specified as `dim3`: `dim3 gridDim(x, y, z)`.
3. For 1D: `dim3 gridDim(n/256, 1, 1)` — enough blocks to cover n elements with 256 threads each.
4. For 2D: `dim3 gridDim((W+15)/16, (H+15)/16, 1)` — covers W x H image with 16×16 blocks.
5. Inside kernel, `gridDim.x` gives total number of blocks in x-direction.
6. Grid can have maximum 65535 blocks per dimension (older GPUs) or $2^{31}-1$ (newer GPUs).
7. Total threads = `gridDim.x * blockDim.x` (for 1D case).

Q9) Kernel in CUDA? Kernel Launch? Arguments in Kernel Launch.

- A Kernel is the core concept of CUDA programming — it is the function that actually runs in parallel on the GPU.
- Understanding kernel definition and launch syntax is the most essential skill in CUDA programming.
- The kernel launch configuration tells the GPU how many threads to create and how to organize them.

What is a Kernel:

1. A kernel is a C function declared with `__global__` qualifier that executes on the GPU.
2. It is called from CPU code (host) but runs on the GPU (device).
3. When launched, one single kernel function is executed by thousands of threads simultaneously.
4. Each thread runs the exact same code but uses its unique thread ID to work on different data.

5. Kernels cannot return values (return type must be void); output is written to device memory.
6. Kernels can call `__device__` functions (GPU helper functions) but cannot call CPU functions.

What is Kernel Launch:

1. Kernel launch is the act of calling the kernel from CPU code to start GPU execution.
2. Uses special triple angle bracket syntax `<<>>` not found in standard C.
3. The CPU sends the launch request to the GPU driver, which schedules thread blocks on SMs.
4. Kernel launch is asynchronous by default — CPU continues executing the next line immediately.
5. To wait for GPU to finish, `cudaDeviceSynchronize()` must be called explicitly.

UNIT 4 — Parallel Sorting, Graph Algorithms & Cloud

Q1) Issues in Sorting on Parallel Computers.

- Sorting is one of the most fundamental operations in computing, but parallelizing it introduces several unique challenges.
- Unlike sequential sorting, parallel sorting requires coordination between processors through communication and synchronization.
- Understanding these issues helps in designing efficient parallel sorting algorithms.

1. Data Distribution

Input data must be divided among processors before sorting. | Uneven distribution = some processors get more work than others. | Poor split wastes processors and increases total execution time. | Requires coordination time just to distribute data.

2. Load Balancing

After partial sorting, some processors finish early and sit idle. | Idle processors = wasted parallel resources. | Most common bottleneck in parallel sorting. | Parallel speedup is rarely equal to number of processors because of this.

3. Communication Overhead

Processors must exchange data during sorting (e.g., merge phases). | Network/bus communication is much slower than local computation. | Too much data movement can make parallel sorting slower than sequential. | Rule: parallel sorting is only beneficial when $T_{\text{compute}} \gg T_{\text{comm}}$.

4. Synchronization

After each phase, processors must wait for each other at a barrier. | If one processor is slow, all others wait — entire pipeline stalls. | Called synchronization overhead — unavoidable idle time. | More processors = more synchronization points = more waiting.

5. Scalability

Adding more processors does not always speed things up proportionally. | Beyond a point, communication and sync costs dominate computation. | Governed by Amdahl's Law — sequential portion limits max speedup. | Example: if 20% of sort is sequential $\rightarrow$ max speedup = 5x, no matter how many processors.

6. Comparison Network Design

Parallel sorting needs a fixed comparison network (e.g., Bitonic Sort). | Deciding which elements to compare and when is non-trivial. | Wrong comparison order $\rightarrow$ output is not sorted. | Network must guarantee a globally sorted result after all stages.

Q2) Odd-Even Transposition in Bubble Sort using Suitable Example.

- Odd-Even Transposition Sort is a parallel version of bubble sort designed for parallel computers.
 - It works in alternating odd and even phases where adjacent processor pairs compare and exchange their values simultaneously.
 - It guarantees sorting in exactly p phases where p is the number of processors.
1. Each processor holds exactly one element and is numbered starting from 1 ($P_1, P_2, P_3, P_4 \dots$).
 2. In each phase, adjacent pairs of processors compare their values and swap if they are out of order.
 3. Odd Phase: Processors at odd positions pair with their right neighbor — pairs (P_1, P_2), (P_3, P_4), (P_5, P_6)...
 4. Even Phase: Processors at even positions pair with their right neighbor — pairs (P_2, P_3), (P_4, P_5), (P_6, P_7)...

5. In each pair, smaller value goes to left processor and larger value goes to right processor.
6. Both odd and even phases happen in parallel — all pairs compare and swap simultaneously.
7. The algorithm always terminates in exactly p phases regardless of input order.
8. After p phases, the array is guaranteed to be globally sorted in ascending order.

Example with 4 processors: Initial = $P_1=3, P_2=1, P_3=4, P_4=2$

Phase 1 (Odd): Pairs (P_1, P_2) and (P_3, P_4) — compare 3,1 $\rightarrow$ swap $\rightarrow$ 1,3; compare 4,2 $\rightarrow$ swap $\rightarrow$ 2,4

After Phase 1: $P_1=1, P_2=3, P_3=2, P_4=4$

Phase 2 (Even): Pair (P_2, P_3) — compare 3,2 $\rightarrow$ swap $\rightarrow$ 2,3

After Phase 2: $P_1=1, P_2=2, P_3=3, P_4=4$ (Sorted!)

Parallel Complexity: Each phase takes $O(1)$ time since all comparisons happen simultaneously.

Total time complexity: $O(p) = O(n)$ for n elements on n processors, compared to $O(n^2)$ for sequential bubble sort.

Speedup: Sequential bubble sort $O(n^2)$ vs Parallel odd-even $O(n)$ — speedup of $O(n)$ times.

This algorithm is simple to implement, requires only neighbor communication, and works well on ring and mesh topologies.

Q3) i) Parallel Merge Sort ii) GPU Applications

- Parallel Merge Sort and GPU Applications are two important topics in high performance computing.
- Parallel Merge Sort applies the divide-and-conquer strategy across multiple processors to speed up sorting.
- GPU Applications demonstrate how GPU parallelism is used to solve real-world compute-intensive problems.

i) Parallel Merge Sort:

1. Parallel Merge Sort is based on the classic merge sort algorithm adapted for execution on multiple processors.
2. The input array of n elements is divided equally among p processors — each processor gets n/p elements.
3. Phase 1 — Local Sort: Each processor independently sorts its local n/p elements using any sequential sort algorithm.
4. All local sorts happen simultaneously in parallel, taking $O((n/p) \log(n/p))$ time.
5. Phase 2 — Parallel Merge (Tree Reduction): Processors merge their sorted lists in a binary tree pattern.
6. Step 1: Pairs of processors merge — P_0 merges with P_1 , P_2 merges with P_3 ($p/2$ merges in parallel).
7. Step 2: Results are merged again — P_0 merges result with P_2 's result ($p/4$ merges in parallel).
8. Parallel Merge Sort achieves good speedup for large arrays and is widely used in practice.

ii) GPU Applications:

1. Deep Learning: Training neural networks — matrix multiplications in forward/backward pass run massively faster on GPU.
2. Image Processing: Applying filters, edge detection, and transformations on millions of pixels in parallel.
3. Scientific Simulations: Weather forecasting, fluid dynamics, molecular dynamics simulations on GPU clusters.
4. Cryptocurrency Mining: SHA-256 hashing for Bitcoin mining leverages GPU parallelism for maximum hash rate.
5. Video Rendering: Real-time 3D rendering in games and animation studios uses GPU parallel processing pipelines.
6. Medical Imaging: CT scan and MRI reconstruction using parallel filtered back-projection algorithms on GPU.

Q4) What is Kubernetes? Features and Applications.

- Kubernetes (also written as K8s) is an open-source container orchestration platform originally developed by Google.
- It automates the deployment, scaling, and management of containerized applications across a cluster of machines.
- Kubernetes has become the industry standard for managing large-scale cloud-native and distributed applications.

Features:

1. Automatic Bin Packing: Kubernetes automatically places containers on nodes based on available CPU and memory resources.
2. Self-Healing: If a container crashes or a node fails, Kubernetes automatically restarts or reschedules the container on another node.
3. Horizontal Scaling: Kubernetes automatically adds or removes container instances based on CPU usage or custom metrics.
4. Load Balancing: Kubernetes distributes incoming network traffic equally across multiple container instances automatically.
5. Automated Rollouts and Rollbacks: New versions are deployed gradually; if something fails, Kubernetes automatically rolls back to the previous version.
6. Service Discovery: Kubernetes assigns DNS names and IP addresses to containers so they can find and communicate with each other automatically.
7. Storage Orchestration: Kubernetes automatically mounts storage systems like local storage, cloud storage (AWS EBS, GCP disks) to containers.
8. Secret and Config Management: Sensitive data like passwords and API keys are stored and managed securely without exposing them in container images.

Applications:

1. Microservices: Running many small independent services together (used by Netflix, Uber, Airbnb).
2. CI/CD Pipelines: Automating build, test, and deployment pipelines for software development teams.
3. HPC Batch Jobs: Managing and scheduling parallel computing jobs in research and scientific computing environments.
4. Cloud Platforms: Managed Kubernetes services — AWS EKS, Google GKE, Azure AKS — power cloud-native applications worldwide.

Q5) Parallel Depth First Search (DFS) Algorithm in Detail.

- Depth First Search (DFS) explores a graph by going as deep as possible along each branch before backtracking.
- Sequential DFS is inherently difficult to parallelize because of its recursive and data-dependent nature.
- Parallel DFS distributes the exploration of different branches across multiple processors simultaneously.

Parallel DFS Algorithm Steps

Step 1 — Initialization:

Mark all nodes as unvisited. | Push start node onto the stack. | Assign start node to Processor 0.

Step 2 — Explore and Split:

Each processor explores its node and finds unvisited neighbors. | If multiple neighbors exist -> split them among available processors. | Each processor gets one branch to explore independently.

Step 3 — Local DFS:

Each processor performs sequential DFS on its assigned branch. | Marks visited nodes in shared visited array. | Uses locking/synchronization to avoid two processors visiting same

node.

Step 4 — Synchronization:

Processors check shared visited array before visiting any node. | If node already visited -> skip it (no duplicate work). | `__syncthreads()` or mutex locks ensure safe access to shared data.

Step 5 — Combine Results:

After all processors finish their branches. | Results (visited order, paths) are merged. | Final DFS traversal order is assembled.

- Applications: Parallel DFS is used in cycle detection, topological sorting, finding connected components, and solving maze problems.
- Limitation: Parallel DFS is less efficient than Parallel BFS for most graph problems due to irregular work distribution and high communication cost.

Q6) Parallel Breadth First Search (BFS) Algorithm in Detail.

- Breadth First Search (BFS) explores a graph level by level, visiting all nodes at distance d before visiting nodes at distance $d+1$.
- Parallel BFS distributes the processing of each frontier level across multiple processors simultaneously.
- It is easier to parallelize than DFS because all nodes at the same level can be processed independently.

Parallel BFS Algorithm — Steps

Step 1 — Initialization:

Mark all nodes as unvisited. | Put start node in frontier level 0. | Assign level 0 frontier to all processors.

Step 2 — Process Current Frontier:

Each processor takes a subset of current frontier nodes. | For each node -> find all unvisited neighbors. | Add unvisited neighbors to the next frontier.

Step 3 — Synchronization:

All processors synchronize after processing each level. | Ensures next frontier is complete before processing begins. | Uses barrier synchronization between levels.

Step 4 — Mark Visited:

Newly discovered nodes marked as visited in shared array. | Use atomic operations to prevent two processors marking same node. | Avoids duplicate entries in next frontier.

Step 5 — Repeat:

Next frontier becomes current frontier. | Repeat Steps 2-4 until frontier is empty. | BFS complete — all reachable nodes visited.

Applications of Parallel BFS

- Shortest path finding — GPS navigation, network routing.
- Social network analysis — finding degrees of separation (LinkedIn, Facebook).
- Web crawling — exploring web pages level by level.
- Game AI — finding nearest goal in puzzle/maze solving.

Challenges in BFS:

- Load imbalance — some levels have very few nodes -> processors sit idle.
- Memory bandwidth — large frontiers require huge memory access -> bottleneck on GPU.
- Irregular memory access — graph edges are stored irregularly -> poor cache performance.

Q7) Distributed Computing for Document Classification.

- Document Classification is the task of automatically assigning categories or labels to text documents.
- Real-world datasets have millions to billions of documents, single CPU cannot process all documents in reasonable

time, distributed systems allow parallel processing across many machines.

- With millions of documents to classify, distributed computing is used to process multiple documents in parallel across many machines.
- This approach significantly reduces classification time and makes it scalable for large datasets like web pages or emails.

Key Components:

1. Master Node:

Controls the entire pipeline. | Splits document corpus into chunks. | Assigns chunks to worker nodes. | Collects and aggregates final results.

2. Worker Nodes:

Receive document chunks from master. | Perform preprocessing + classification independently. | Send results back to master. | Can be added/removed dynamically for scalability.

Processing Pipeline — Step by Step

Step 1 — Data Ingestion:

Raw documents collected from sources — web, emails, databases. | Stored in HDFS (Hadoop Distributed File System) or cloud storage. | Documents partitioned into equal-sized chunks across nodes.

Step 2 — Preprocessing (parallel on each worker):

Tokenization — split text into words/tokens. | Stop word removal — remove 'the', 'is', 'and' etc. | Stemming/Lemmatization — reduce words to root form. | TF-IDF / Word Embeddings — convert text to numerical features.

Step 3 — Model Distribution:

Pre-trained classification model loaded on every worker. | Workers use same model — ensures consistent classification. | Model can be Naive Bayes, SVM, or Deep Learning (BERT).

Step 4 — Parallel Classification:

Each worker classifies its chunk independently. | No communication needed between workers during classification. | Results stored locally on each worker.

Step 5 — Result Aggregation:

Master node collects all results from workers. | Merges into final classified dataset. | Stores output back to HDFS/database.

Q8) i) Random ii) Ring iii) Blackboard Communication Strategy

i) Random Communication Strategy:

- In Random Communication Strategy, each processor sends messages to randomly selected processors rather than following a fixed pattern.
- There is no predetermined structure — any processor can communicate with any other processor at random.
- This strategy is simple and requires no topology knowledge or routing tables.
- It naturally provides load balancing because work is spread randomly across all processors.
- Used in randomized parallel algorithms, gossip protocols, and distributed hash tables.
- Advantage: Simple to implement, avoids bottlenecks, works well for irregular communication patterns.
- Disadvantage: Communication may be inefficient — messages may travel long distances, and some processors may receive too many messages while others receive too few.

ii) Ring Communication Strategy:

- In Ring Communication Strategy, processors are arranged in a logical ring — each processor communicates only with its left and right neighbors.
- Data passes around the ring in one direction (or both directions in bidirectional ring) step by step.
- Used in all-to-all broadcast on rings — each processor passes received data to the next processor.
- Communication is structured and predictable — easy to implement and analyze.
- Total steps required = $p-1$ where p is the number of processors.
- Used in parallel matrix multiplication (Cannon's algorithm), circular shift operations, and token-passing protocols.
- Advantage: Simple topology, low complexity, each processor only needs to know its two neighbors.
- Disadvantage: Slow for large p because data must travel through many intermediate processors — latency = $O(p)$.

iii) Blackboard Communication Strategy:

- In Blackboard Communication Strategy, all processors share a common global memory space called the blackboard (or shared memory).
- Any processor can read from or write to the blackboard at any time — similar to a shared whiteboard.
- No direct processor-to-processor messaging is needed — communication happens through the shared blackboard.
- Used in knowledge-based systems, expert systems, parallel problem solving, and multi-agent AI systems.
- Synchronization is required — processors must use locks or semaphores to prevent simultaneous conflicting writes.
- Knowledge Sources (processors) independently check the blackboard, perform computation, and post results back.
- Used in parallel BFS where visited array acts as a shared blackboard that all processors read and update.
- Advantage: Flexible and easy to coordinate — any processor can share results with all others instantly by writing to blackboard.
- Disadvantage: Blackboard can become a bottleneck when many processors try to access it simultaneously — limits scalability.

Q9) Communication Strategies for Parallel BFS.

- In Parallel BFS, the main challenge is efficiently communicating frontier vertices between processors at each level.
- Different communication strategies offer different trade-offs between communication cost, synchronization, and scalability.
- Choosing the right strategy is critical for achieving good parallel BFS performance on large graphs.

Communication Strategies for Parallel BFS

Strategy 1 — Shared Memory Communication

All processors share a common visited array and frontier queue. | Processors read/write directly to shared global memory. | Atomic operations prevent two processors marking same node simultaneously. | After each level -> barrier sync before moving to next level. | Used in multi-core CPU and GPU systems.

Strategy 2 — Message Passing Communication (MPI)

Each processor owns a local partition of the graph. | When a neighbor node belongs to another processor -> send a message. | Uses MPI_Send, MPI_Recv, MPI_Alltoall to exchange frontier nodes. | After each level -> all-to-all exchange to share newly discovered nodes. | Used in

distributed memory clusters.

Strategy 3 — Bulk Synchronous Parallel (BSP) Model

Computation split into supersteps — each superstep has 3 phases: Compute -> each processor processes its local frontier. | Communicate -> send cross-partition discovered nodes to owners. | Synchronize -> global barrier, wait for all messages to arrive. | Repeat supersteps until no more nodes discovered. | Used in Google Pregel, Apache Giraph for web-scale graphs.

Strategy 4 — Asynchronous Communication

Processors do NOT wait for others to finish before continuing. | No barrier synchronization between levels. | Processors process nodes and send messages independently. | Messages arrive and are processed whenever they are ready. | Opposite of BSP/barrier-based strategies.

Strategy 5 — Work Stealing

A dynamic load balancing technique for parallel BFS. | Each processor manages its own local frontier queue. | If a processor's queue becomes empty -> it actively steals work from a busy processor. | Ensures no processor sits idle as long as work exists anywhere in the system.